

1. Write a program to make a parent process wait for child process

C

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        // Child process
        printf("Child process (PID: %d) executing...\n", getpid());
        sleep(2); // Simulate work
        printf("Child process exiting.\n");
        exit(0);
    } else if (pid > 0) {
        // Parent process
        printf("Parent process (PID: %d) waiting for child (PID: %d)...\n", getpid(), pid);
        wait(NULL); // Wait for child to finish
        printf("Parent process: Child has finished. Exiting.\n");
    } else {
        perror("fork failed");
        exit(1);
    }
    return 0;
}
```

Sample Output:

text

```
Parent process (PID: 1234) waiting for child (PID: 1235)...
Child process (PID: 1235) executing...
Child process exiting.
Parent process: Child has finished. Exiting.
```

2. Write a program to create a zombie process

C

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        // Child process
        printf("Child (PID: %d) exiting immediately.\n", getpid());
        exit(0);
    } else if (pid > 0) {
        // Parent process
        printf("Parent (PID: %d) sleeping. Child becomes zombie.\n", getpid());
        sleep(30); // Keep parent alive to observe zombie (use 'ps' in another terminal)
        printf("Parent exiting. Zombie reaped.\n");
    } else {
        perror("fork failed");
    }
    return 0;
}
```

Sample Output:

text

Parent (PID: 1234) sleeping. Child becomes zombie.

Child (PID: 1235) exiting immediately.

Parent exiting. Zombie reaped.

3. Write a program to create a child process different from parent process (Orphan process)

C

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
int main() {  
    pid_t pid = fork();  
  
    if (pid == 0) {  
        // Child process  
        sleep(5); // Child sleeps longer  
        printf("Orphan child (PID: %d, PPID: %d) running after parent exit.\n", getpid(), getppid());  
    } else if (pid > 0) {  
        // Parent process  
        printf("Parent (PID: %d) exiting immediately.\n", getpid());  
    } else {  
        perror("fork failed");  
    }  
    return 0;  
}
```

Sample Output:

text

Parent (PID: 1234) exiting immediately.

Orphan child (PID: 1235, PPID: 1) running after parent exit.

4. Write a program to illustrate creation of thread using POSIX threads

C

```
#include <stdio.h>
#include <pthread.h>

void* thread_func(void* arg) {
    int id = *(int*)arg;
    printf("Thread %d running.\n", id);
    return NULL;
}

int main() {
    pthread_t threads[3];
    int ids[3] = {1, 2, 3};

    for (int i = 0; i < 3; i++) {
        pthread_create(&threads[i], NULL, thread_func, &ids[i]);
    }

    for (int i = 0; i < 3; i++) {
        pthread_join(threads[i], NULL);
    }
    printf("All threads finished.\n");
    return 0;
}
```

Sample Output:

text

```
Thread 1 running.
Thread 2 running.
Thread 3 running.
All threads finished.
```

5. Write a program to make the main process to wait for the child process to complete

C

// Same as Q1 (using waitpid for specific child)

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
int main() {
```

```
    pid_t pid = fork();
```

```
    if (pid == 0) {
```

```
        printf("Child working...\n");
```

```
        sleep(1);
```

```
        printf("Child done.\n");
```

```
        exit(0);
```

```
    } else {
```

```
        int status;
```

```
        waitpid(pid, &status, 0); // Wait specifically for this child
```

```
        printf("Main (parent) process: Child completed.\n");
```

```
    }
```

```
    return 0;
```

```
}
```

Sample Output:

text

Parent process (PID: 1234) waiting for child (PID: 1235)...

Child process (PID: 1235) executing...

Child process exiting.

Parent process: Child has finished. Exiting.

6. Write a program to illustrate sharing of variables between threads and main process

C

```
#include <stdio.h>
#include <pthread.h>

int global_var = 0;

void* thread_func(void* arg) {
    global_var += 10;
    printf("Thread: global_var = %d\n", global_var);
    return NULL;
}

int main() {
    pthread_t t;
    printf("Main before thread: global_var = %d\n", global_var);
    pthread_create(&t, NULL, thread_func, NULL);
    pthread_join(t, NULL);
    printf("Main after thread: global_var = %d\n", global_var);
    return 0;
}
```

Sample Output:

text

```
Main before thread: global_var = 0
Thread: global_var = 10
Main after thread: global_var = 10
```

7. Write a program to illustrate how a single function can handle more than one process (signals)

C

```
#include <stdio.h>
```

```
#include <signal.h>
```

```
#include <unistd.h>
```

```
void signal_handler(int sig) {
```

```
    if (sig == SIGINT) printf("Caught SIGINT (Ctrl+C)\n");
```

```
    else if (sig == SIGTERM) printf("Caught SIGTERM\n");
```

```
}
```

```
int main() {
```

```
    struct sigaction sa;
```

```
    sa.sa_handler = signal_handler;
```

```
    sigemptyset(&sa.sa_mask);
```

```
    sa.sa_flags = 0;
```

```
    sigaction(SIGINT, &sa, NULL);
```

```
    sigaction(SIGTERM, &sa, NULL);
```

```
    printf("Waiting for signals... (Ctrl+C or kill -15 %d)\n", getpid());
```

```
    while(1) pause();
```

```
    return 0;
```

```
}
```

Sample Output:

text

Waiting for signals...

Caught SIGINT (Ctrl+C)

8. FCFS Scheduling

C

```
#include <stdio.h>
```

```
int main() {
    int n = 4;
    int bt[] = {5, 8, 12, 6}; // Burst times
    int wt[4], tat[4];
    float avg_wt = 0, avg_tat = 0;

    wt[0] = 0;
    for (int i = 1; i < n; i++) wt[i] = wt[i-1] + bt[i-1];

    for (int i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
        avg_wt += wt[i];
        avg_tat += tat[i];
    }

    printf("Process\tBurst\tWait\tTurnaround\n");
    for (int i = 0; i < n; i++) printf("P%d\t%d\t%d\t%d\n", i+1, bt[i], wt[i], tat[i]);

    printf("Avg Wait: %.2f\nAvg TAT: %.2f\n", avg_wt/n, avg_tat/n);
    return 0;
}
```

Sample Output:

text

Process	Burst	Wait	Turnaround
P1	5	0	5
P2	8	5	13
P3	12	13	25
P4	6	25	31

Avg Wait: 10.75
Avg TAT: 18.50

9. SJF (Non-preemptive)

C

```
#include <stdio.h>
```

```
struct Process {  
    int id, bt, wt, tat;  
};
```

```
int main() {  
    struct Process p[4] = {{1,5}, {2,8}, {3,12}, {4,6}};  
    // Sort by burst time (simple bubble sort)  
    for (int i = 0; i < 3; i++)  
        for (int j = 0; j < 3-i; j++)  
            if (p[j].bt > p[j+1].bt) {  
                struct Process temp = p[j];  
                p[j] = p[j+1];  
                p[j+1] = temp;  
            }  
  
    p[0].wt = 0;  
    for (int i = 1; i < 4; i++) p[i].wt = p[i-1].wt + p[i-1].bt;  
    for (int i = 0; i < 4; i++) p[i].tat = p[i].wt + p[i].bt;  
  
    printf("Process\tBurst\tWait\tTurnaround\n");  
    for (int i = 0; i < 4; i++) printf("P%d\t%d\t%d\t%d\n", p[i].id, p[i].bt, p[i].wt, p[i].tat);  
    return 0;  
}
```

The program outputs the following table:

text

Process	Burst	Wait	Turnaround
P1	5	0	5
P4	6	5	11
P2	8	11	19
P3	12	19	31

10. Round Robin (Quantum = 4)

C

```
#include <stdio.h>
```

```
int main() {
    int n = 4, quantum = 4;
    int bt[] = {5, 8, 12, 6};
    int rem_bt[4];
    for (int i = 0; i < n; i++) rem_bt[i] = bt[i];

    int t = 0, wt[4] = {0}, tat[4];
    while (1) {
        int done = 1;
        for (int i = 0; i < n; i++) {
            if (rem_bt[i] > 0) {
                done = 0;
                if (rem_bt[i] > quantum) {
                    t += quantum;
                    rem_bt[i] -= quantum;
                } else {
                    t += rem_bt[i];
                    wt[i] = t - bt[i];
                    rem_bt[i] = 0;
                }
            }
        }
        if (done) break;
    }

    for (int i = 0; i < n; i++) tat[i] = bt[i] + wt[i];
    printf("Process\tBurst\tWait\tTurnaround\n");
    for (int i = 0; i < n; i++) printf("P%d\t%d\t%d\t%d\n", i+1, bt[i], wt[i], tat[i]);
    return 0;
}
```

The program outputs:

Process	Burst	Wait	Turnaround
P1	5	15	20
P2	8	18	26
P3	12	19	31
P4	6	10	16

11. Dining Philosophers

C

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5

sem_t chopstick[N];

void* philosopher(void* num) {
    int id = *(int*)num;
    while (1) {
        printf("Philosopher %d thinking...\n", id);
        sleep(1);

        sem_wait(&chopstick[id]);
        sem_wait(&chopstick[(id+1)%N]);

        printf("Philosopher %d eating...\n", id);
        sleep(1);

        sem_post(&chopstick[id]);
        sem_post(&chopstick[(id+1)%N]);
    }
    return NULL;
}

int main() {
    pthread_t phil[N];
    int ids[N];
    for (int i = 0; i < N; i++) sem_init(&chopstick[i], 0, 1);

    for (int i = 0; i < N; i++) {
        ids[i] = i;
        pthread_create(&phil[i], NULL, philosopher, &ids[i]);
    }

    for (int i = 0; i < N; i++) pthread_join(phil[i], NULL);
    return 0;
}
```

Expected Output (Typical Run) Philosopher 0 thinking...Philosopher 1 thinking...Philosopher 2 thinking...Philosopher 3 thinking...Philosopher 4 thinking...

12. Producer-Consumer (Bounded Buffer)

C

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <semaphore.h>
```

```

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0, out = 0;
sem_t empty, full, mutex;

void* producer(void* arg) {
    for (int i = 0; i < 10; i++) {
        sem_wait(&empty);
        sem_wait(&mutex);
        buffer[in] = i;
        printf("Produced: %d\n", i);
        in = (in + 1) % BUFFER_SIZE;
        sem_post(&mutex);
        sem_post(&full);
    }
    return NULL;
}

void* consumer(void* arg) {
    for (int i = 0; i < 10; i++) {
        sem_wait(&full);
        sem_wait(&mutex);
        int item = buffer[out];
        printf("Consumed: %d\n", item);
        out = (out + 1) % BUFFER_SIZE;
        sem_post(&mutex);
        sem_post(&empty);
    }
    return NULL;
}

int main() {
    pthread_t prod, cons;
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);
    return 0;
}

```

```
}
```

Sample Output (Interleaved, exact order varies by thread scheduling)

A typical run produces output like this:

```
Produced: 0
Produced: 1
Produced: 2
Produced: 3
Produced: 4
Consumed: 0
Produced: 5
Consumed: 1
Produced: 6
Consumed: 2
Produced: 7
Consumed: 3
Produced: 8
Consumed: 4
Produced: 9
Consumed: 5
Consumed: 6
Consumed: 7
Consumed: 8
Consumed: 9
```

13-14. Readers-Writers Problem

C

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
```

```

sem_t wrt, mutex;
int readcount = 0;

void* reader(void* arg) {
    sem_wait(&mutex);
    readcount++;
    if (readcount == 1) sem_wait(&wrt);
    sem_post(&mutex);

    printf("Reader reading... (readers: %d)\n", readcount);
    sleep(1);

    sem_wait(&mutex);
    readcount--;
    if (readcount == 0) sem_post(&wrt);
    sem_post(&mutex);
    return NULL;
}

void* writer(void* arg) {
    sem_wait(&wrt);
    printf("Writer writing...\n");
    sleep(1);
    sem_post(&wrt);
    return NULL;
}

int main() {
    pthread_t r[5], w[2];
    sem_init(&wrt, 0, 1);
    sem_init(&mutex, 0, 1);

    // Create some readers and writers
    for (int i = 0; i < 5; i++) pthread_create(&r[i], NULL, reader, NULL);
    for (int i = 0; i < 2; i++) pthread_create(&w[i], NULL, writer, NULL);

    for (int i = 0; i < 5; i++) pthread_join(r[i], NULL);
    for (int i = 0; i < 2; i++) pthread_join(w[i], NULL);
    return 0;
}

```

output (one common execution trace) is:

```

Reader reading... (readers: 1)
Reader reading... (readers: 2)

```

Reader reading... (readers: 3)

Reader reading... (readers: 4)

Reader reading... (readers: 5)

Writer writing...

Writer writing...